Data Structures for the Banker's Algorithm

Let n = number of processes, and m = number of resources types.

- **Available**: Vector of length m. If available $[j] = k$, there are k instances of resource type R_j available
- **Max**: n x m matrix. If $Max\,[i,j] = k$, then process T_i may request at most k instances of resource type R_j
- **Allocation**: n x m matrix. If Allocation$[i,j] = k$ then T_i is currently allocated k instances of R_j
- **Need**: n x m matrix. If $Need[i,j] = k$, then T_i may need k more instances of R_j to complete its task

$$Need\,[i,j] = Max[i,j] – Allocation\,[i,j]$$

Safety Algorithm

1. Let ***Work*** and ***Finish*** be vectors of length *m* and *n*, respectively. Initialize:

   Work* = *Available

   ***Finish* [*i*] = *false* for *i* = 0, 1, …, *n*- 1**

2. Find an ***i*** such that both:

   (a) ***Finish* [*i*] = *false***

   (b) $\boldsymbol{Need_i \leq Work}$

   If no such ***i*** exists, go to step 4

3. $\boldsymbol{Work = Work + Allocation_i}$

   Finish*[*i*] = *true

   go to step 2

4. If ***Finish* [*i*] == *true*** for all ***i***, then the system is in a safe state

Resource-Request Algorithm for Process P_i

$\boldsymbol{Request_i}$ = request vector for process $\boldsymbol{T_i}$. If $\boldsymbol{Request_i}$ **[*j*] = *k*** then process $\boldsymbol{T_i}$ wants ***k*** instances of resource type $\boldsymbol{R_j}$

1. If $\boldsymbol{Request_i} \leq \boldsymbol{Need_i}$ go to step 2. Otherwise, raise error condition, since process has exceeded its maximum claim
2. If $\boldsymbol{Request_i} \leq \boldsymbol{Available}$, go to step 3. Otherwise $\boldsymbol{T_i}$ must wait, since resources are not available
3. Pretend to allocate requested resources to $\boldsymbol{T_i}$ by modifying the state as follows:

$$\boldsymbol{Available = Available - Request_i;}$$

$$\boldsymbol{Allocation_i = Allocation_i + Request_i;}$$

$$\boldsymbol{Need_i = Need_i - Request_i;}$$

- If safe $\Rightarrow$ the resources are allocated to $\boldsymbol{T_i}$
- If unsafe $\Rightarrow$ $\boldsymbol{T_i}$ must wait, and the old resource-allocation state is restored

Example of Banker's Algorithm

- 5 threads T_0 through T_4;

  3 resource types:

  A (10 instances), *B* (5instances), and *C* (7 instances)

- Snapshot at time X

	Allocation	*Max*	*Available*
	A B C	*A B C*	*A B C*
T_0	0 1 0	7 5 3	3 3 2
T_1	2 0 0	3 2 2	
T_2	3 0 2	9 0 2	
T_3	2 1 1	2 2 2	
T_4	0 0 2	4 3 3	

what each thread has

how many of each available right now

need = max − allocation

ex.

T_0

$need_A = 7-0=7$

$need_B = 5-1=4$

$need_C = 3-0=3$

T_3

$need_A = 2-2=0$

$need_B = 2-1=1$

$need_C = 2-1=1$

Example (Cont.)

- The content of the matrix ***Need*** is defined to be ***Max – Allocation***

Available

A	B	C
3	3	2

	Need
	A B C
T_0	7 4 3
T_1	1 2 2
T_2	6 0 0
T_3	0 1 1
T_4	4 3 1

either can be first

another solution

* pick 1st available $\langle T_1, T_3, T_0, T_2, T_4 \rangle$

- The system is in a safe state since the sequence $< T_1, T_3, T_4, T_2, T_0 >$ satisfies safety criteria

T_1

Available

A	B	C
5	3	2

T_3 could do T_4

A	B	C
7	4	3

T_0 can do T_2 or T_4 too

A	B	C
7	5	3

T_2

A	B	C
6	5	5

T_4

A	B	C
10	5	7

new request

Example: T_1 Request (1,0,2)

request

- Check that Request ≤ Available (that is, (1,0,2) ≤ (3,3,2) ⇒ true

Prev. available

	Allocation	*Need*	*Available*
	A B C	*A B C*	*A B C*
T_0	0 1 0	7 4 3	2 3 0
T_1	3 0 2	0 2 0	
T_2	3 0 2	6 0 0	
T_3	2 1 1	0 1 1	
T_4	0 0 2	4 3 1	

safe, so update available

Old: Alloc
T_1 2 0 0
need
1 2 2
request 1,0,2
done

- Executing safety algorithm shows that sequence < T_1, T_3, T_4, T_0, T_2> satisfies safety requirement
- Can request for (3,3,0) by T_4 be granted? → NO (3,3,0) ≰ (2,3,0)
- Can request for (0,2,0) by T_0 be granted? → yes (0,2,0) ≤ (2,3,0)